P4-SpecTec

A Language Mechanization Framework for P4

Jaehyun Lee, Seokhun Jeong, Sukyoung Ryu
@ HATRA 2025

A DSL for the Networking Domain

A Language Mechanization Framework

Adopting Language Mechanization Approach into P4

Adopting Language Mechanization Approach into P4

What is mechanization, and **why** do we need it?

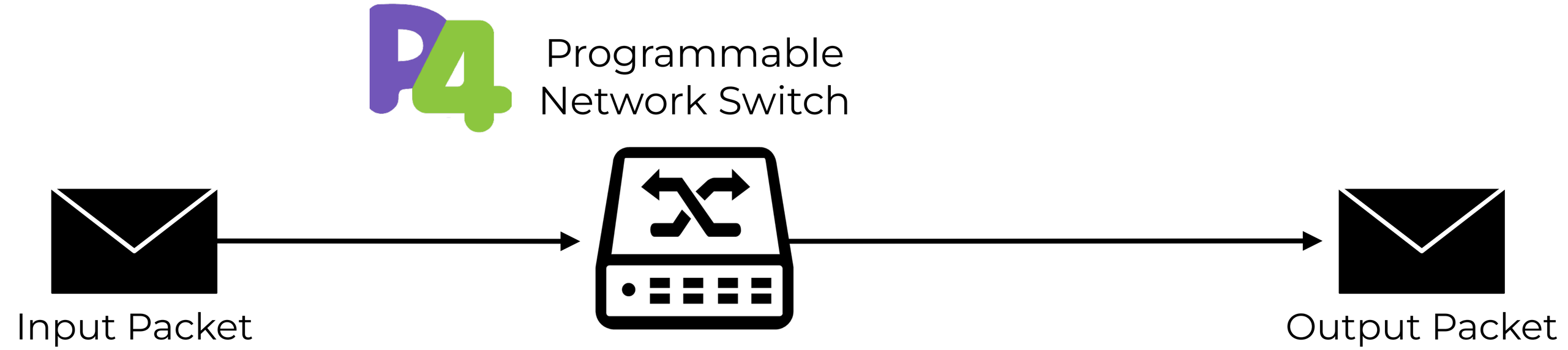
How do we mechanize the P4 language specification?

What are the benefits?

The P4 Programming Language

Programming Protocol-independent Packet Processors (P4)

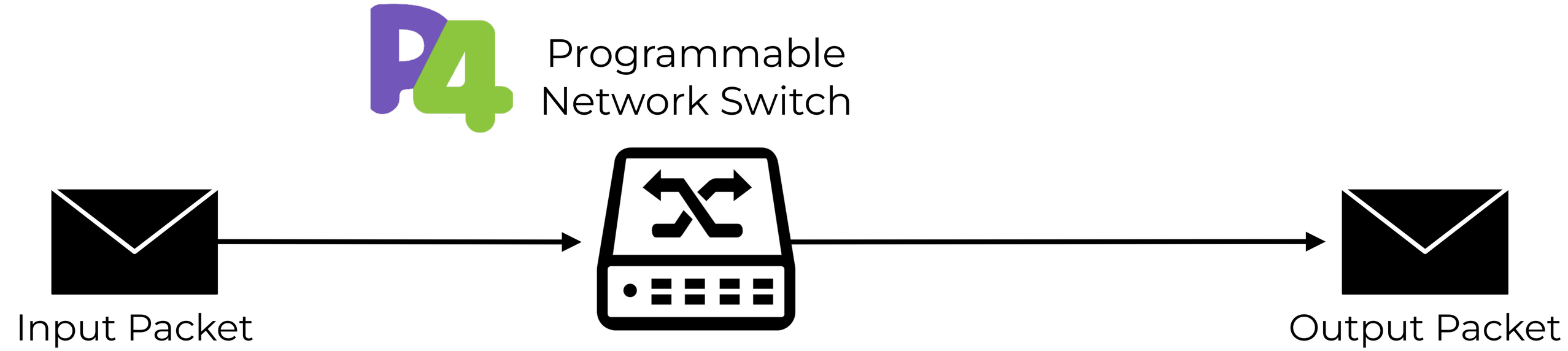
- DSL for network devices (switches)
- Specify how to take in, process, and emit a network packet



The P4 Programming Language

Programming Protocol-independent Packet Processors (P4)

- ✓ Statically typed
- ✓ Imperative like the C language

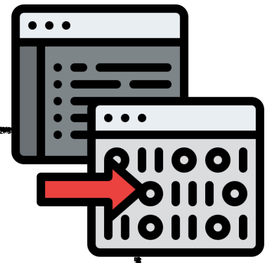


PL POV: Four representations of the P4 language

informal prose

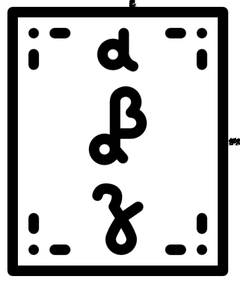
Official Spec

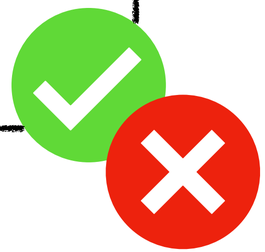
p4c reference compiler in C++



Implementations

Formalizations





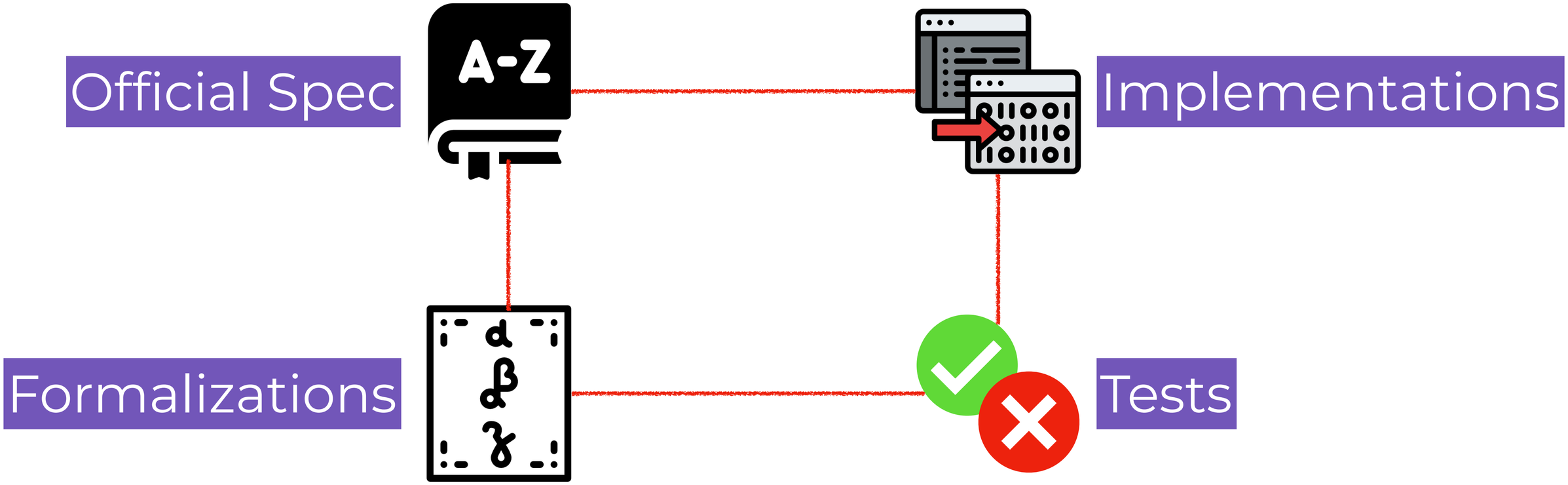
Tests

precise mathematical notations

input-output examples

Are the four representations truly in sync?

i.e., are they **consistent** with one another? in principle, YES; in practice, NO



Maintained by different parties, and evolve at different paces

Part of the Reason: P4 is Evolving

Official Spec Introduced generic types @ May 2021

A.3. Summary of changes made in version 1.2.2, released May 17, 2021

- Added support for accessing tuple fields (Section 8.12).
- Added support for generic structures (Section 7.2.11).

Implementations Generics + Type Inference? Patched @ March 2025

Compiler Bug: `Could not find type of <Type_Header> ...` on specialized generic struct type #4835

Closed Bug #5133

Formalizations Remains pre-2021, the recent version is based 2024

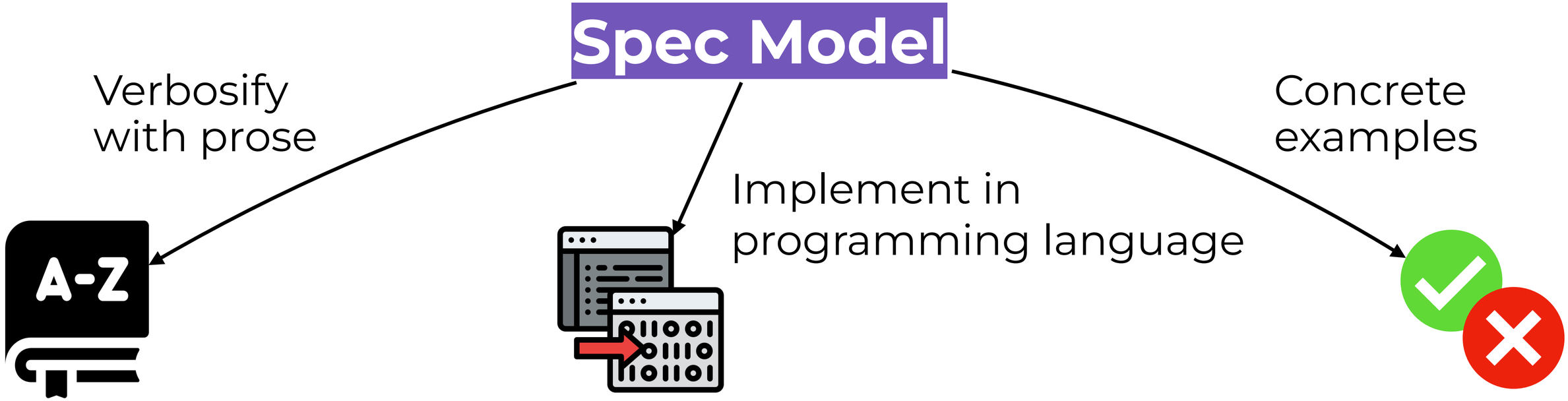
Can we, at the same time,
evolve a programming language,
and maintain the consistency between representations?

Single Source of Truth Underlying the Representations

The **Specification Model**,

an unambiguous and complete definition of the syntax and semantics.

e.g., complete formalization, definitional interpreter

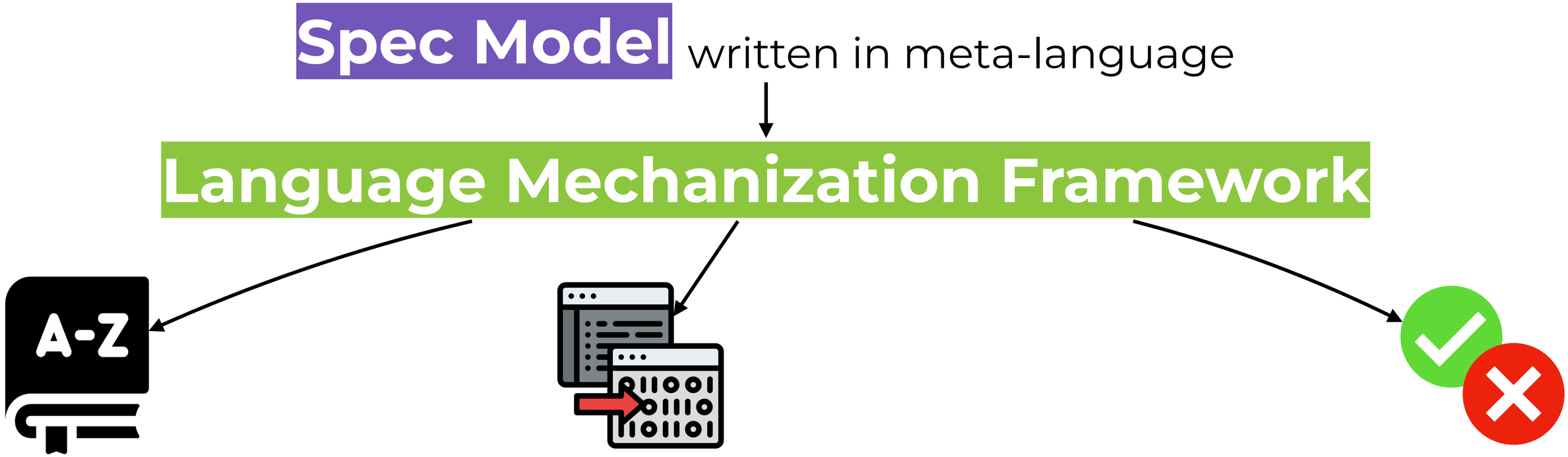


Language Mechanization Frameworks for Consistency

Language mechanization frameworks

provide a DSL (called meta-language) for defining the spec model

and automate generation of various backends from it



P4 - SpecTec

Adopting Language Mechanization Approach into P4

What is mechanization, and **why** do we need it?

How do we mechanize the P4 language specification?

What are the benefits?

Dongjun Youn, Wonho Shin, Jaehyun Lee, Sukyoung Ryu, Joachim Breitner, Philippa Gardner, Sam Lindley, Matija Pretnar, Xiaojia Rao, Conrad Watt, and Andreas Rossberg. 2024. Bringing the WebAssembly Standard up to Speed with SpecTec. Proc. ACM Program. Lang. 8, PLDI, Article 210 (June 2024), 26 pages. https://doi.org/10.1145/3656440

Mechanization Integrated in Real-World Languages

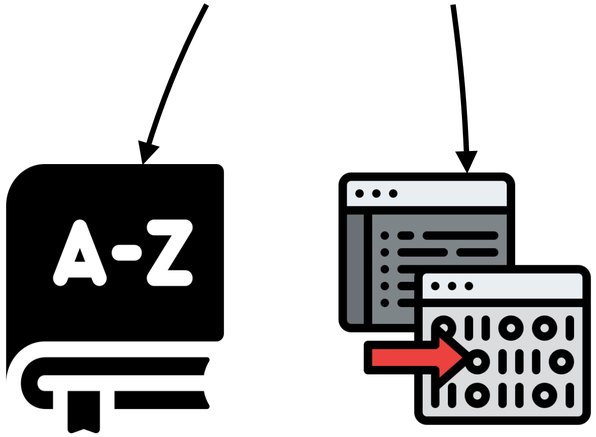
WA

Wasm-SpecTec

SpecTec has been adopted

Published on March 27, 2025 by Andreas Rossberg.

https://webassembly.org/news/2025-03-27-spectec/



Prose Spec step-by-step pseudocode-style explanation

if bt $instr_1^*$ $instr_2^*$

1. Assert: Due to validation, a value of number type i32 is on the top of the stack.
2. Pop the value (i32.const c) from the stack.
3. If $c \neq 0$, then:
    a. Execute the instruction (block bt $instr_1^*$).
4. Else:
    a. Execute the instruction (block bt $instr_2^*$).

$$
\begin{array}{lcll}
(\text{i32.const } c)\ (\text{if } bt\ instr_1^*\ \text{else}\ instr_2^*) & \hookrightarrow & (\text{block } bt\ instr_1^*) & \text{if } c \neq 0 \\
(\text{i32.const } c)\ (\text{if } bt\ instr_1^*\ \text{else}\ instr_2^*) & \hookrightarrow & (\text{block } bt\ instr_2^*) & \text{if } c = 0
\end{array}
$$

Formal Spec dynamic semantics as formal reduction rules

The Wasm-SpecTec Spec Model

$$(\text{i32.const } c)\ (\text{if } bt\ instr_1^*\ \text{else}\ instr_2^*) \hookrightarrow (\text{block } bt\ instr_1^*) \quad \text{if } c \neq 0$$
$$(\text{i32.const } c)\ (\text{if } bt\ instr_1^*\ \text{else}\ instr_2^*) \hookrightarrow (\text{block } bt\ instr_2^*) \quad \text{if } c = 0$$

Resemble in ASCII characters

Spec Model Reduction rules described with SpecTec metalang

```
rule Step_pure/if-true:
  (CONST I32 c) (IF bt instr_1* ELSE instr_2*) ~> (BLOCK bt instr_1*)
  -- if c =/= 0

rule Step_pure/if-false:
  (CONST I32 c) (IF bt instr_1* ELSE instr_2*) ~> (BLOCK bt instr_2*)
  -- if c = 0
```

Wasm-SpecTec Prose Backend

Spec Model

Declarative reduction rules ("What")

```
rule Step_pure/if-true: ...
rule Step_pure/if-false: ...
```

Prose Spec

Algorithmic pseudocode

("How")

1. Assert: Due to validation, a value of number type i32 is on the top of the stack.
2. Pop the value (i32.const c) from the stack.
3. If $c \neq 0$, then:
    a. Execute the instruction (block bt $instr_1^*$).
4. Else:
    a. Execute the instruction (block bt $instr_2^*$).

Declarative to Algorithmic

```
LHS ~> RHS
-- Premise(s)
```

```
1. Pop ... from the stack.
2. Let ... be ...
3. If ...,
    A. Push ... to the stack.
```

"What" reduction takes place

"How" to compute the reduction

Non-trivial in general;

Assume a shared format: reduction rules on a stack machine

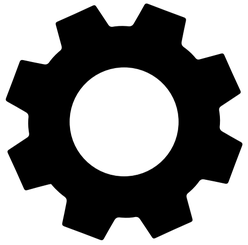
Wasm-SpecTec Interpreter Backend

Prose algorithms express the Wasm semantics

```
LHS ~> RHS
-- Premise(s)
```

```
1. Pop ... from the stack.
2. Let ... be ...
3. If ...,
    A. Push ... to the stack.
```

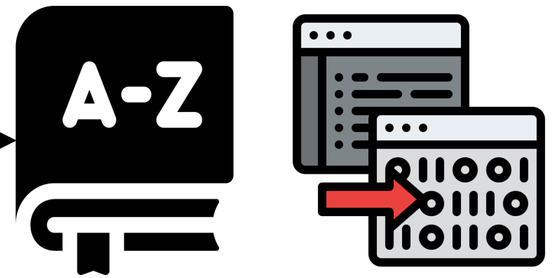
Execute the prose algorithms with a prose interpreter



Wasm Interpreter

Wasm-SpecTec, Integrated into the Wasm Spec

"Two weeks ago, the Wasm Community Group voted to adopt SpecTec for authoring future editions of the Wasm spec."

Spec Model → Wasm-SpecTec → 

Edit the spec model,

using Wasm-SpecTec metalang

Underlying Recipe: An Authoritative Spec Model

THE standard

$$(\textbf{i32.const}\ c)\ \textbf{if}\ bt\ instr_1^*\ \textbf{else}\ instr_2^*\ \textbf{end} \quad \hookrightarrow \quad \textbf{block}\ bt\ instr_1^*\ \textbf{end} \quad (\text{if } c \neq 0)$$

$$(\textbf{i32.const}\ c)\ \textbf{if}\ bt\ instr_1^*\ \textbf{else}\ instr_2^*\ \textbf{end} \quad \hookrightarrow \quad \textbf{block}\ bt\ instr_2^*\ \textbf{end} \quad (\text{if } c = 0)$$

is the starting point for the spec model design

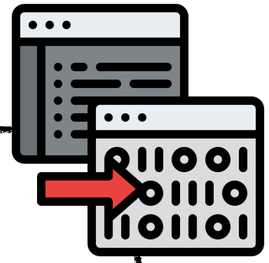
Formal Reduction Rules

P4 Lacks an Authoritative Spec Model

Informal, not systematic

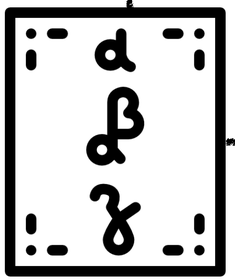
Nano-pass compiler

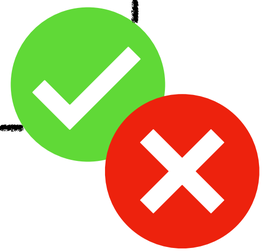
Official Spec



Implementations

Formalism





Tests

Incomplete, not up-to-date

Incomplete

Can we reuse Wasm-SpecTec meta-language for P4?

Declarative Inference (Reduction) Rules

- Hide implementation-level details
- Inference rules are not directly executable

Can handle Wasm dynamic semantics but,

```
rule Type_sub/trans:
  C |- type_a <: type_c
 -- Type_sub: C |- type_a <: type_b
 -- Type_sub: C |- type_b <: type_c
```

cannot handle static semantics, *cannot* generate a type checker

For P4: Algorithmic Inference Rules

Declarative Inference Rules

Abstract, yet not directly executable

Algorithmic Inference Rules

Subset of inference rules that are constructive

Wasm-SpecTec meta-language with constraints

P4 if Statement Specification

12.6. Conditional statement

The conditional statement uses standard syntax and semantics familiar from many programming languages.

However, the condition expression in P4 is required to be a Boolean (and not an integer).

P4 syntax in P4-SpecTec

```
syntax stmt =
  | IF `( expr ) stmt ELSE stmt
  | ...
```

Algorithmic typing rule in P4-SpecTec

```
relation Stmt_ok:
  context |- stmt : context


rule Stmt_ok/ifStmt:
  C |- IF `( expr_c ) stmt_t ELSE stmt_f
      : C_res
  -- Expr_ok: C |- expr_c : type_c
  -- if BOOL = type_c
  -- Stmt_ok: C |- stmt_t : C_t
  -- Stmt_ok: C |- stmt_f : C_f
  -- if C_res = C
```

With intuitive sense,

- Subject of the rule is:
  - C, and
  - IF `(` expr_c `)` stmt_t ELSE stmt_f
- Result of the rule is: C_res
- Computation/data flows from subject to result

```
relation Stmt_ok:
  context |- stmt : context

rule Stmt_ok/ifStmt:
   C |- IF `( expr_c ) stmt_t ELSE stmt_f
        : C_res
   -- Expr_ok: C |- expr_c : type_c
   -- if BOOL = type_c
   -- Stmt_ok: C |- stmt_t : C_t
   -- Stmt_ok: C |- stmt_f : C_f
   -- if C_res = C
```

Constraining the SpecTec meta-language

Constraint 1. Annotate input/outputs of relations

```
relation Stmt_ok:
  context |- stmt : context (hint input %0 %1)

rule Stmt_ok/ifStmt:
  C |- IF `( expr_c ) stmt_t ELSE stmt_f : C_res
  -- ...
```

Constrain the interpretation of mathematical relations;

Relations in P4-SpecTec have designated inputs and outputs

- Input: C, IF `(expr_c) stmt_t ELSE stmt_f
- Output: C_res

Constraining the SpecTec meta-language

Constraint 2. Data flows sequentially through inputs, premises, outputs

```
rule Stmt_ok/ifStmt:
  C |- IF `( expr_c ) stmt_t ELSE stmt_f : C_res
  -- Expr_ok: C |- expr_c : type_c
  -- if BOOL = type_c
  -- Stmt_ok: C |- stmt_t : C_t
  -- Stmt_ok: C |- stmt_f : C_f
  -- if C_res = C
```

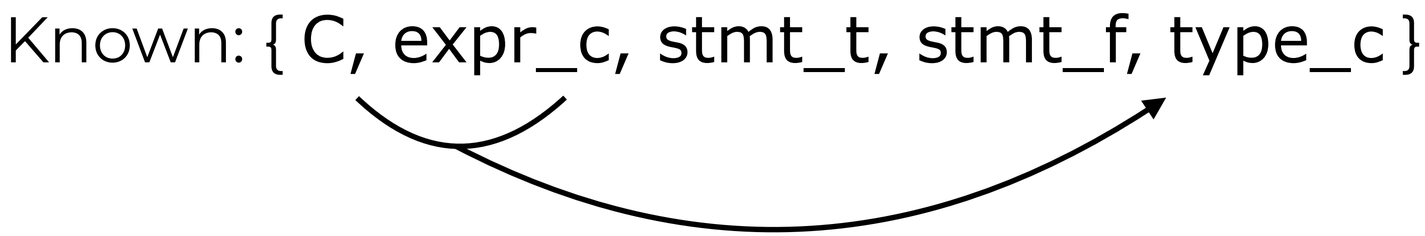
Known: { C, expr_c, stmt_t, stmt_f }

Constraining the SpecTec meta-language

Constraint 2. Data flows sequentially through inputs, premises, outputs

```
rule Stmt_ok/ifStmt:
  C |- IF `( expr_c ) stmt_t ELSE stmt_f : C_res
  -- Expr_ok: C |- expr_c : type_c
  -- if BOOL = type_c
  -- Stmt_ok: C |- stmt_t : C_t
  -- Stmt_ok: C |- stmt_f : C_f
  -- if C_res = C
```

```
relation Expr_ok:
  context |- expr : type
  (hint input %0 %1)
```



Constraining the SpecTec meta-language

Constraint 3. Except binding, all variables must be known beforehand

```
rule Stmt_ok/ifStmt:
  C |- IF `( expr_c ) stmt_t ELSE stmt_f : C_res
  -- Expr_ok: C |- expr_some : type_c
  -- if BOOL = type_c
  -- Stmt_ok: C |- stmt_t : C_t
  -- Stmt_ok: C |- stmt_f : C_f
  -- if C_res = C
```

```
relation Expr_ok:
  context |- expr : type
  (hint input %0 %1)
```

expr_some is unknown; Forbids existential premises

Constraining the SpecTec meta-language

Constraint 4. if x = y can mean either: (i) x == y (ii) x := y (iii) y := x

```
rule Stmt_ok/ifStmt:
  C |- IF `( expr_c ) stmt_t ELSE stmt_f : C_res
  -- Expr_ok: C |- expr_c : type_c
  -- if BOOL = type_c
  -- Stmt_ok: C |- stmt_t : C_t
  -- Stmt_ok: C |- stmt_f : C_f
  -- if C_res = C
```

Disambiguate the overloaded `=`

- ✓ Both sides are known: x == y
- ✓ Only RHS is known: x := y
- ✓ Only LHS is known: y := x

Constraining the SpecTec meta-language

Constraint 4. if x = y can mean either: (i) x == y (ii) x := y (iii) y := x

```
rule Stmt_ok/ifStmt:
  C |- IF `( expr_c ) stmt_t ELSE stmt_f : C_res
  -- Expr_ok: C |- expr_c : type_c
  -- if BOOL = type_c
  -- Stmt_ok: C |- stmt_t : C_t
  -- Stmt_ok: C |- stmt_f : C_f
  -- if C_res = C
```

Known: { C, expr_c, stmt_t, stmt_f, type_c }

So, a comparison `==`

Constraining the SpecTec meta-language

Constraint 2. Data flows sequentially through inputs, premises, outputs

```
rule Stmt_ok/ifStmt:
  C |- IF `( expr_c ) stmt_t ELSE stmt_f : C_res
  -- Expr_ok: C |- expr_c : type_c
  -- if BOOL = type_c
  -- Stmt_ok: C |- stmt_t : C_t
  -- Stmt_ok: C |- stmt_f : C_f
  -- if C_res = C
```

```
relation Stmt_ok:
  context |- stmt : context
  (hint input %0 %1)
```

Known: { C, expr_c, stmt_t, stmt_f, type_c, C_t, C_f }

Constraining the SpecTec meta-language

Constraint 4. if x = y can mean either: (i) x == y (ii) x := y (iii) y := x

```
rule Stmt_ok/ifStmt:
  C |- IF `( expr_c ) stmt_t ELSE stmt_f : C_res
  -- Expr_ok: C |- expr_c : type_c
  -- if BOOL = type_c
  -- Stmt_ok: C |- stmt_t : C_t
  -- Stmt_ok: C |- stmt_f : C_f
  -- if C_res = C
```

Known: { C, expr_c, stmt_t, stmt_f, type_c, C_t, C_f }

So, an assignment C_res := C

P4 Spec Model written in meta-language

↓

P4-SpecTec

Constraint 1. Annotate relation input/outputs

Constraint 2. Data flows through inputs, premises, outputs

Constraint 3. All variables must be known except in binders

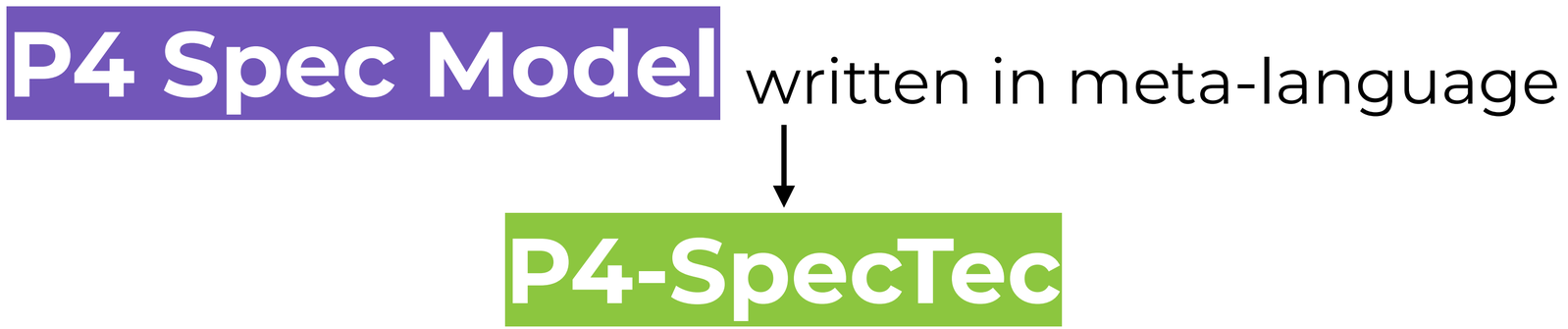
Constraint 4. `=` is disambiguated into `==`, `:=`, or `=:`

... and more ...

The P4 Spec Model

Full P4 spec model of syntax and type system

~ 8K LoC in meta-language



P4 - SpecTec

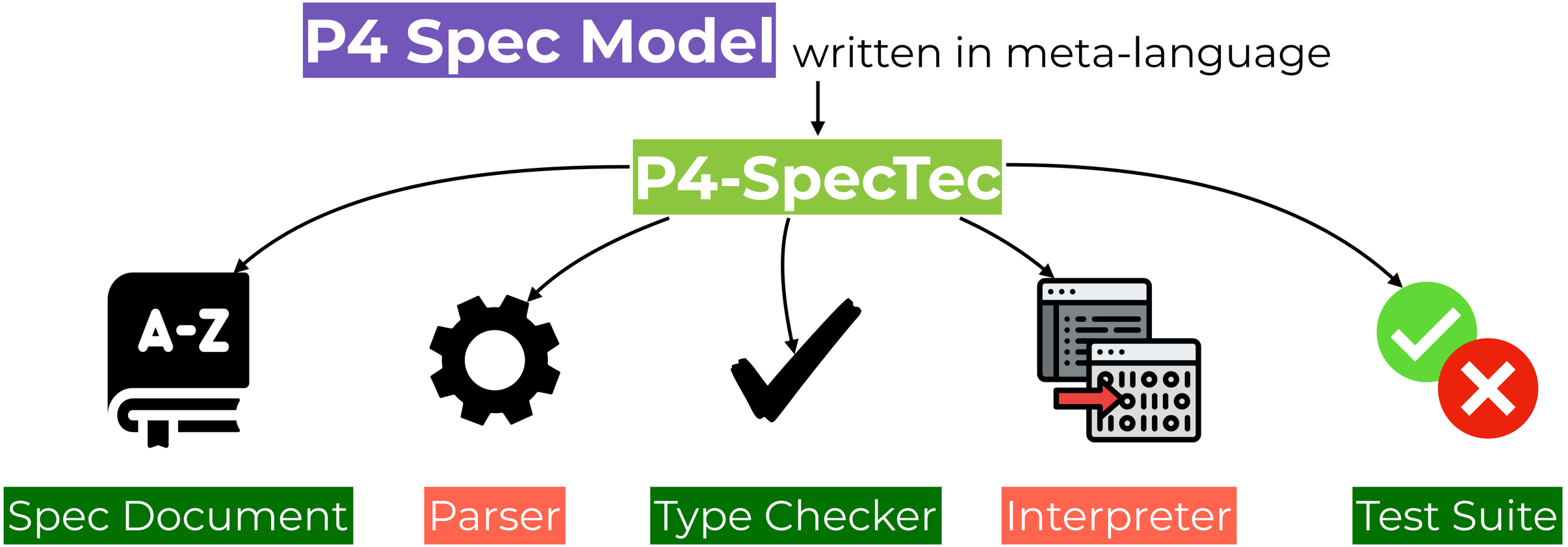
Adopting Language Mechanization Approach into P4

What is mechanization, and **why** do we need it?

How do we mechanize the P4 language specification?

What are the benefits?

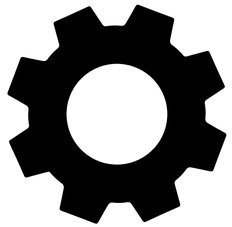
P4-SpecTec Vision and Current Status



P4-SpecTec Type Checker Backend

P4 Spec Model

```
relation Stmt_ok:
  context |- stmt : context
  (hint input %0 %1)
```



Metalang Interpreter

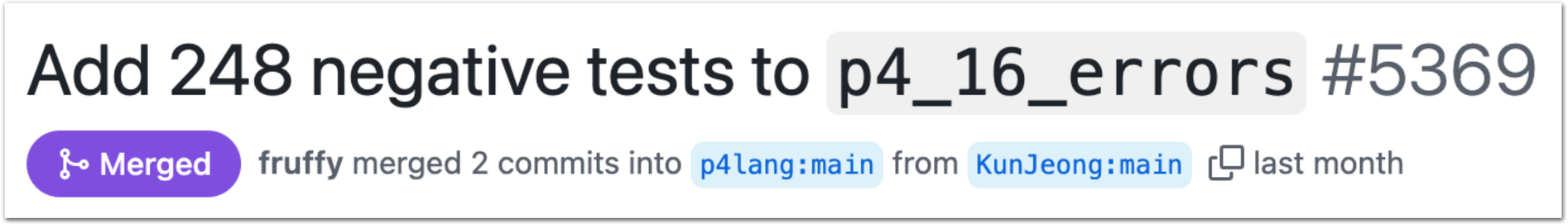
= P4 Type Checker

Validated with the p4c reference compiler test suite:

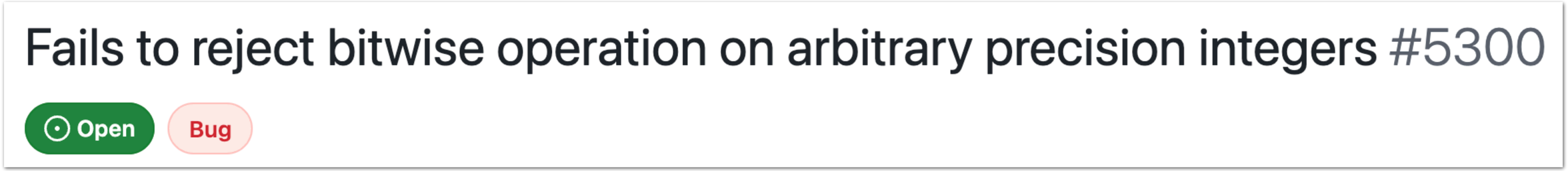
- Positive tests: 1063/1067 (99.6%)
- Negative tests: 254/258 (98.4%)

P4-SpecTec Negative Type Checker Test Backend

Added negative type tests for previously untested type errors



Discovered 26 previously unknown bugs in the p4c compiler frontend



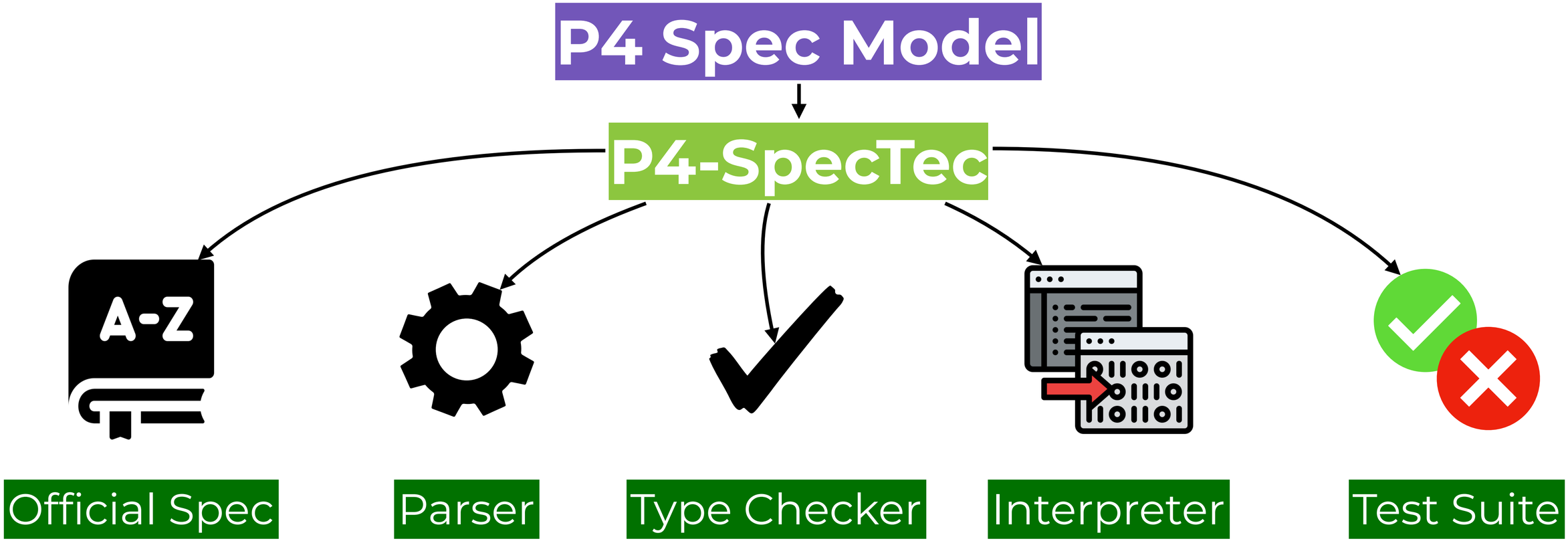
P4-SpecTec Specification Document Backend

`rule Stmt_ok/ifStmt: ...` → Render as prose instructions in HTML

Typing `IF ( expr_c ) stmt_t ELSE stmt_f` under typing context `C`:

1. Let `type_c` be the result of typing `expr_c` under typing context `C`.
2. Check that `type_c` matches pattern `BOOL`.
3. Let `C_t` be the result of typing `stmt_t` under typing context `C`.
4. Let `C_f` be the result of typing `stmt_f` under typing context `C`.
5. Result in the typing context `C`.

(1) Edit the mechanized spec



(2) Execute the spec, validating that existing tests pass

(3) Auto-generate new tests for the edits

(4) Auto-generate the spec document in correspondence

P4 - SpecTec

Language Design Working Group

Mailing List

Github

Co-Chairs

- Jonathan DiLorenzo (Google)
- Ryan Goodfellow (Oxide Computer Company)

Adopting Language Mechanization Approach into P4

Supplementary Slides

p4c Test Pass Rate

[Total] 1,278 Positive tests + 310 Negative tests

[Exclusion]

- X Experimental features (33)
- X Specific features to some compile targets (59)
- X Under discussion with the LDWG / p4c developers (68)

[Fails]

- X Parser bug
- X Timeout due to large integer literals
- X ...

Disallowing Existential Premises (More)

Binding is only allowed within invertible constructs

```
rule Stmt_ok/ifStmt:
  C |- IF `( expr_c ) stmt_t ELSE stmt_f : C_res
  -- Expr_ok: C |- expr_c : $func_some(type_c)
  -- if BOOL = type_c
  -- Stmt_ok: C |- stmt_t : C_t
  -- Stmt_ok: C |- stmt_f : C_f
  -- if C_res = C
```

type_c should be some inverse image of $func_some; non-constructive

only allow binders within invertible, e.g., Expr_ok: C |- expr : (VarT id)

Disallowing Non-Determinism

Typing rules match sequentially;

comes with a consequence -- the model is sequential, not concurrent

Subtyping without Existential Premises

Unroll the transitivity

```
rule Subtype/bool-int32:
 BOOL <: INT32
rule Subtype/int32-int64:
 INT32 <: INT64
rule Subtype/trans:
 type_a <: type_c
 -- Subtype: type_a <: type_b
 -- Subtype: type_b <: type_c
```

```
rule Subtype/bool-int32:
 BOOL <: INT32
rule Subtype/int32-int64:
 INT32 <: INT64
rule Subtype/bool-int64:
 BOOL <: INT64
```

But luckily, P4 subtyping is simple

How do we generate the prose?

Annotate relations with prose *hints*

```
relation Stmt_ok:
  context ⊢ stmt : context
  hint(input %0 %1)
  hint(prose_in "typing" %1 "under typing context" %0)
  hint(prose_out "the typing context" %2)
```